

Projet: Génération de maillage pour le calcul scientifique

Antoine Levrat

03/2026

Projet partie 1 : *L'objectif de ce travail est d'implémenter des fonctions qui mesurent de qualité géométrique d'une triangulation et de développer une structure de table de hachage performante pour gérer la connectivité des éléments.*

Structure du maillage : Avant toute chose, il convient de s'intéresser aux différents éléments d'un maillage. Ici on forme un tableau `Crd`, de coordonnées. Ainsi `Crd[iVer][0]` donne la coordonnée du sommet `iVer`.

Puis un tableau `Tri` qui répertorie les numéros des sommets. La colonne `i` à la ligne `j` contient les numéros du sommet de l'arête `j` du triangle `i`.

`Tri2Edge` est une table de correspondance qui permet de récupérer le numéro local d'un sommet e.g) `[Tri2Edge[iEdg][1]]` donne le numéro local du deuxième sommet de `iEdg`.

Enfin `TriVoi`, qui va être un sujet important de ce projet est une table qui répertorie pour chaque triangle ses trois voisins adjacents. Plus précisément, `TriVoi[iTri][iEdg]` stocke le numéro du triangle qui partage l'arête `iEdg` avec le triangle `iTri`.

1 Qualité des éléments

Pour quantifier la régularité géométrique des triangles au sein du maillage, nous avons commencé par implémenter deux fonctions :

1. $Q_1(K) = \alpha_1 \frac{\sum_i l_i^2}{|K|}$
2. $Q_2(K) = \alpha_2 \frac{h_{max}}{\rho(K)}$

Où l_i représente la longueur des arêtes, h_{max} la plus grande arête, et $\rho(K)$ le rayon du cercle inscrit. Les constantes α_1 et α_2 sont calculées pour obtenir une valeur de 1 pour un triangle équilatéral. Pour ce triangle équilatéral de côté a , l'aire est $|K| = \frac{\sqrt{3}}{4}a^2$.

Pour Q_1 , on veut $Q_1 = 1$, donc $\alpha_1 \frac{3a^2}{\frac{\sqrt{3}}{4}a^2} = 1 \implies \alpha_1 = \frac{\sqrt{3}}{12}$.

Pour Q_2 , avec $\rho = \frac{a\sqrt{3}}{6}$ et $h_{max} = a$, on a $\alpha_2 \frac{a}{\frac{a\sqrt{3}}{6}} = 1 \implies \alpha_2 = \frac{\sqrt{3}}{6}$.

Ces fonctions, dont les valeurs oscillent entre 1 et $+\infty$ et les résultats sont donnés ci-dessous :

Comparaison de la qualité du maillage d'un carré (carre_4h) pour les fonctions Q_1 et Q_2

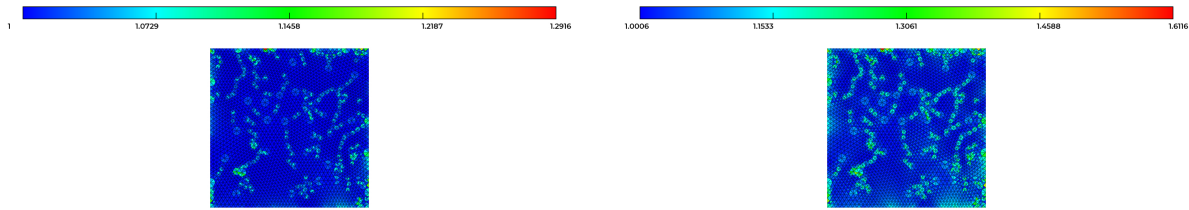


Figure 1 – Etude de Q_1

Figure 2 – Etude de Q_2

Les fonctions données peuvent osciller entre 1 et $+\infty$. Dans le cadre de ce maillage (carre_4h), on voit sur ces visualisations VIZIR que les triangles sont tous de bonne qualité.

2 Introduction d'une table de Hachage

La construction d'un maillage nécessite un tableau de voisins (déterminer quels triangles partagent une arête donnée). Une méthode simple consiste à boucler 2 fois sur le nombre de triangles et 2 fois sur le nombre d'arêtes. Mais cette méthode présente une complexité quadratique $O(n^2)$, et est une approche impraticable car trop longue pour de grand maillages.

Pour optimiser ces recherches, nous avons mis en place une table de hachage caractérisée par :

1. Un tableau de tête (Head) et un tableau de lien (LstObj) pour stocker les objets en collision.
2. Des fonctions pour initialiser, ajouter et trouver des éléments de la table.

Cette structure permet d'accéder rapidement aux objets via une clé, limitant le parcours de recherche à un très petit nombre d'éléments en collision. Cette table a ensuite été utilisée Pour identifier les arêtes frontières et calculer le nombre total d'arêtes.

Nous avons dans un premier temps comparé le temps CPU de l'approche "simple" avec la table de hachage :

On observe bien que le temps de l'approche standard est bien supérieur au temps avec table de Hachage. On peut aussi ajouter que les temps observés pour l'approche lente sont les mêmes que ceux vu en cours. Ils suivent bien comme annoncé une complexité de $O(n^2)$ et $O(h)$.

3 Calcul des arêtes frontières et du nombre total d'arêtes

Grâce à la fonction `hash_add`, nous avons pu recenser les arêtes et leurs triangles adjacents. Une arête est définie comme frontière si elle n'est associée qu'à un seul triangle (le champ du deuxième triangle dans LstObj reste nul). Cette méthode a permis d'extraire les caractéristiques suivantes :

L'efficacité de la table repose sur la qualité de la clé de hachage pour minimiser les collisions. Nous avons expérimenté plusieurs stratégies pour générer une clé unique par arête définie par les sommets i et j .

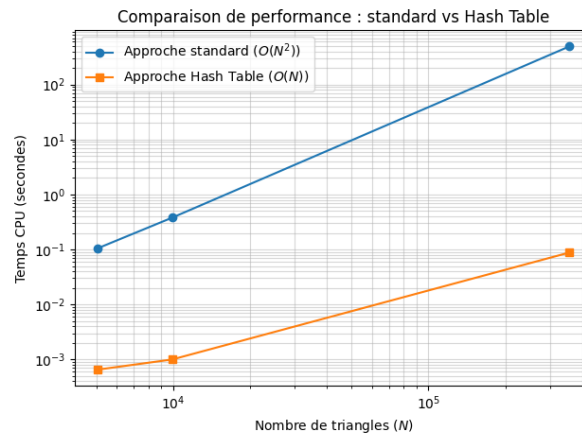


Figure 3 – Comparaison de performance pour un maillage de ≈ 5000 à 355000 triangles

Nous avons donc comparé avec 3 clés : $i+j$, $\min(i,j)$, $i+j*1000$ sur 3 maillages différents dont voici les spécificités :

Maillage	Triangles	Sommets	Arêtes uniques	Arêtes frontières
naca	14266	7379	21645	492
ls89	4098	2239	6337	380
hlcrm	4616	2651	7269	690

Figure 4 – Spécificités des maillages étudiés

	Collision Max $i+j$	Collision Moy $i+j$	Collision Max $\min(i,j)$	Collision Moy $\min(i,j)$	Collision Max $i+j*1000$	Collision Moy $i+j*1000$
naca	15	1.95	6	2.95	8	1.94
ls89	10	2.19	8	3.32	9	1.93
hlcrm	10	2.13	8	3.07	7	1.97

Figure 5 – Comparaison des clés pour la table de Hachage

Nous observons ainsi que le nombre de collisions moyen reste faible. 2 éléments peuvent facilement avoir la même clé l'idée de la troisième clé, $i + j * 1000$ est de réduire ce risque. Elle semble être la plus efficace pour réduire les collisions moyennes.

4 Conclusion préliminaire de la partie 1

Tout au long de ce travail, nous avons implémentés des fonctions pour voir la qualité d'une triangulation et des algorithmes efficaces pour avoir la structure de ce maillage. L'implémentation de table de Hachage c'est avérée capitale pour étudier rapidement des maillages de grande taille. En étudiant la connectivité via une clé de hachage, nous avons pu identifier les arêtes frontières et construire la structure de voisinage en un temps quasi-linéaire.

Projet partie 2 : *L'objectif de ce travail est d'implémenter un algorithme de Delaunay pour une triangulation 2D et l'appliquer à un cas de compression d'image. Certains résultats et codes se basent sur la partie 1 du projet effectué précédemment.*

Nous allons d'abord nous placer dans le carré test $[0, 1]^2$. L'objectif dans un premier temps est de placer des points aléatoirement dans ce carré et les rajouter à la triangulation.

5 Noyau simplifié

Ce premier algorithme se base sur le "edge flip". Il consiste à rajouter un point P , trouver à quel triangle il appartient, créer 3 triangles soit 3 nouvelles arêtes ayant pour sommet le nouveau point P . Puis pour chaque côté de chaque triangle créé, tester la qualité des triangles avec la fonction qualité Q1 ou Q2 du premier projet. On effectue ensuite un edge flip. On recalcule la qualité, en comparant avec les valeurs précédentes on garde la configuration qui avait la meilleure qualité.

On notera qu'il est important de faire attention à bien réactualiser les voisins, sur ces étapes. De plus nous avons choisi, comme dans le sujet srand(7) pour avoir une génération de point reproductible.

Note d'implémentation : La fonction edge flip ne marche que quand les 2 triangles forment un convexe. Pour vérifier ceci, dans un quadrilatère (A, B, C, D) , pour une arête (B, C) on vérifie que le point D est à gauche de (A, B) et le point C à gauche de (A, D) *i.e*) les aires signées de (A, B, D) et (A, D, C) sont positives. (Un dessin permet de s'en assurer)

Par un raisonnement similaire, pour trouver le triangle dans lequel est P on cherche le seul triangle du maillage où l'aire signée est positive pour tous les côtés. On implémente d'abord un algorithme simple qui teste tous les triangles, puis par la suite on a implémenté un algorithme qui suit les directions négatives.

On peut donc visualiser les résultats obtenus : Sur un carré $[0, 1]^2$ avec 50 points insérés aléatoirement via srand(7)

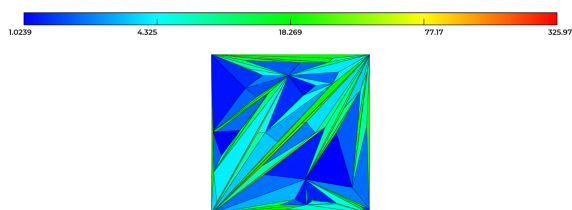


Figure 6 – Sans Edge Flip

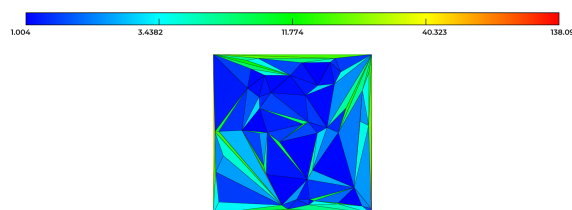


Figure 7 – Avec Edge Flip

la solution montre la qualité Q1 pour chaque triangle. Les images sont visualisées avec VIZIR4, en appliquant une échelle log sur la palette.

On observe que le flip permet d'améliorer grandement la qualité des triangles.

6 Noyau de Delaunay

On essaie maintenant de mettre en place un noyau de Delaunay. L'algorithme se met en place via plusieurs étapes :

1. Localiser le triangle dans lequel est situé le nouveau point P .

2. Calculer la cavité associée à P et en retirer les triangles intérieurs.
3. Créer les triangles en étoilant autour de P .

Pour savoir quels triangles il faut retirer, on effectue le critère de la sphère : *on test : le point P appartient-il au cercle inscrit du triangle testé ?*

Nous avons dans un premier temps testé ce critère sur tout les triangles du maillages pour simplifier l'implémentation. Par la suite, on s'est reservi de la fonction qui localise P , on récupère ensuite les voisins du triangle trouvé, on effectue le test de la sphère sur chaqu'un, et on réitère sur leurs voisins jusqu'à ce que tout les voisins respectent le critère.

Vis à vis de la suppression des triangles on a choisi de récupérer les arrêtes frontières de la cavité, puis d'étoiler P . On réutilise les anciens emplacements des anciens triangles et on en recrée si besoin.

On donne ici les resultats dans la même configuration que dans les Figures précédentes.
i.e) Sur un carré $[0, 1]^2$ avec 50 points insérés aléatoirement via `rand(7)`

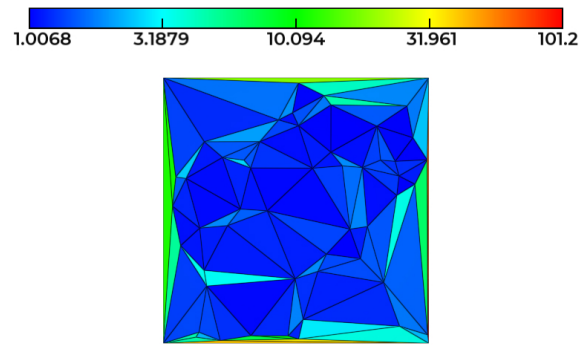


Figure 8 – Noyau de Delaunay

On visualise ici aussi le log de la qualité Q1 de chaque triangle. On voit que la qualité des triangles créés est bien meilleure restant inférieurs à 3 pour la grande majorité des triangles créés.

Par ailleurs on décide de comparer le temps CPU entre les 2 noyau :

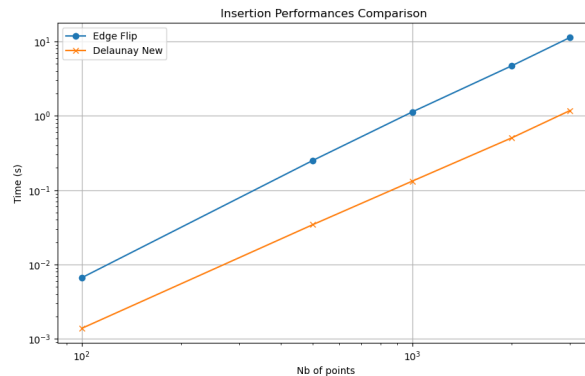


Figure 9 – Comparaison du temps par rapport au nombre de points inserés pour noyau Edge Flip et Delaunay

Le graphique compare en échelle logarithmique le temps passé pour un nombre de points entre 100 et 3000. On observe que pour 3000 le Edge Flip passe 11 secondes, tandis que Delaunay passe un peu plus de 2 secondes.

7 Application à la compression d'image

L'objectif de cette partie est de réduire le maillage nécessaire pour une image tout en préservant sa qualité. Pour ce faire, nous passons d'un maillage structuré régulier, à un maillage non-structuré de Delaunay.

En utilisant l'opérateur de cavité pour insérer des points, nous avons d'abord utilisé une approche aléatoire, puis une méthode adaptative.

7.1 Approche aléatoire

Pour la compression, on charge un maillage ainsi que la valeur de gris de chaque sommet sur ce maillage. Ici on a choisi d'utiliser les fichiers donnés : joconde.mesg et joconde.sol.

On initialise ensuite un maillage initial de 4 sommets de la bonne hauteur/largeur. Par la suite on utilise l'opérateur de delaunay pour des points aléatoires comme dans la partie précédente.

Joconde compressé aléatoirement avec un noyau de delaunay

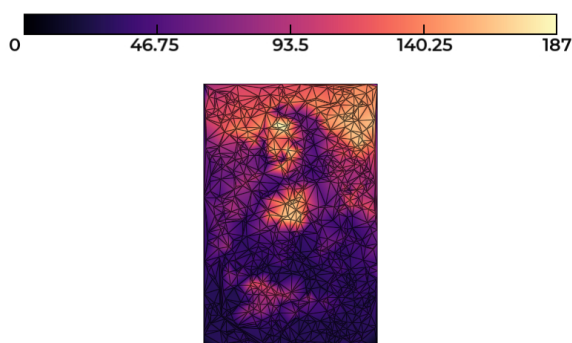


Figure 10 – $N = 1000$ points

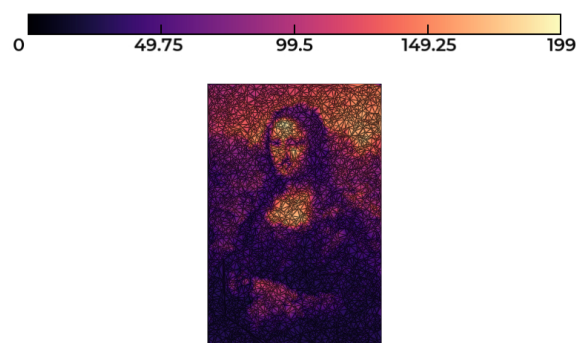


Figure 11 – $N = 5000$ points

On observe ainsi que la triangulation marche correctement, pour 1000 et 5000 points on retrouve la joconde validant ainsi cette approche initiale. On veut ensuite améliorer la qualité de la compression.

Calcul du PSNR : On introduit ensuite une fonction pour calculer le PSNR (*Peak Signal to Noise Ratio*), qui permet de mesurer la qualité de la compression en comparant l'image reconstruite à l'image originale.

Le PSNR est défini par la formule suivante :

$$PSNR = 10 \cdot \log_{10} \left(\frac{d^2}{q_c} \right)$$

où :

- d est l'amplitude maximale du signal (soit $d = 255$ pour une image noir et blanc).
- q_c représente l'erreur quadratique moyenne entre l'image originale u et l'image compressée u_c :

$$q_c = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n |u(i, j) - u_c(i, j)|^2$$

Ici ce pose la question de comment comparer une solution sur un maillage a une autre solution sur un autre maillage. Pour chaque sommet (x, y) sur le maillage ordonné, on trouve a quel triangle il appartient sur le maillage en construction grâce a la fonction développée précédemment pour Delaunay.

Il faut alors interpoler le niveau de gris au point (x, y) pour obtenir la valeur compressée correspondante u_c . Pour ceci, on crée une fonction utilisant les coordonnées barycentriques β_k du pixel dans le triangle K qui le contient :

$$u_c(i, j) = \sum_{k=0}^2 \beta_k u(i_k, j_k)$$

Après avoir implémenté cette fonction, on obtient pour l'image prrécédente (Joconde, 5000 points) un PSNR de l'ordre de 28 décibels.

7.2 Approche PSNR

L'ultime étape consiste à localiser et à insérer d'abord les pixels présentant la plus forte erreur d'interpolation. Cette stratégie permet de densifier le maillage dans les zones de forts contrastes, optimisant ainsi le rapport entre le taux de compression et la fidélité de l'image (mesurée par le PSNR).

Le code implémente cette approche en faisant une boucle itérative qui raffine le maillage localement. À chaque itération, l'algorithme parcourt l'ensemble des pixels de l'image originale pour calculer l'écart absolu $|u(i, j) - u_c(i, j)|$. Le pixel identifié comme ayant l'erreur maximale est alors sélectionné pour être inséré dans le maillage de Delaunay.

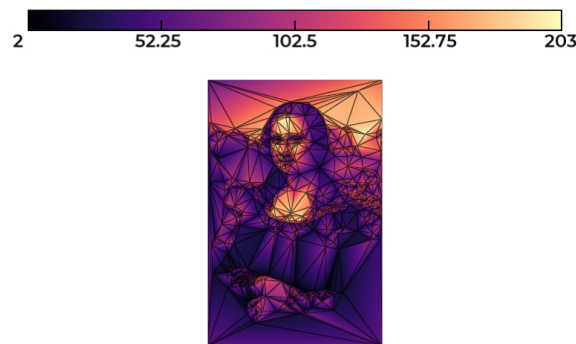


Figure 12 – Joconde compressé via PSNR avec un noyau de delaunay pour $N = 1000$ points

On observe que l'image est bien plus précise même avec un nombre de points limité. Les détails comme les yeux, la bouche, les mains sont biens mieux restitués, meme par rapport à l'image aléatoire avec 5 fois plus de points.

La compression prends cependant un temps bien supérieur a l'approche initiale. Dans nos tests, pour 1000 points aléatoires insérés le temps CPU est inférieur a une seconde, tandis que l'approche via PSNR pour le même nombre de points en prends plutot 25.

Note et améliorations : Les différentes fonctions implémentées dans cette deuxième partie, actualisent les voisins en recalculant l'entièrete du tableau via la table de hachage. Une optimisation possible aurai été de modifier la table en cours de route, en ajoutant/supprimant des lignes au fils des besoins.

8 Conclusion :

Cette étude a permis de comprendre la génération et l'adaptation de maillage. Pour ce faire nous sommes passés de la quantification la qualité des triangles à l'application concrète de la compression d'image. L'implémentation d'une table de hachage s'est avérée important pour gérer la connectivité et réduire la complexité algorithmique de $O(n^2)$ à un temps quasi-linéaire. Par la suite, le passage d'un noyau simplifié par "edge flip" à un noyau de Delaunay a permis d'optimiser la régularité des triangles et les performances de traitement. Enfin, l'application à la compression d'image a démontré la supériorité de l'approche adaptative par PSNR. En densifiant le maillage uniquement dans les zones de forts contrastes, cette méthode restitue les détails avec une précision bien plus élevée que l'approche aléatoire, même avec cinq fois moins de points.